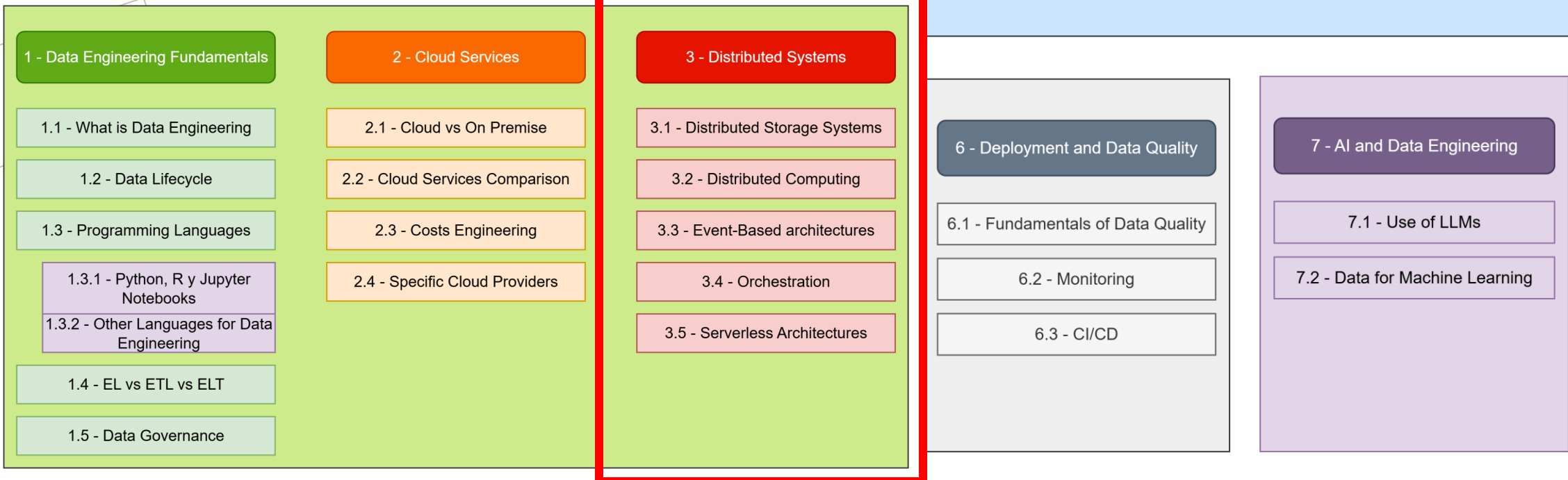


Artificial Intelligence Degree at USJ

DATA ENGINEERING: DISTRIBUTED SYSTEMS

SUBJECT CURRICULUM



DISTRIBUTED SYSTEMS

"A distributed system is a collection of independent computers that appears to its users as a single coherent system. In Spanish, we call it 'Sistema Distribuido.'"

CLUSTERS:

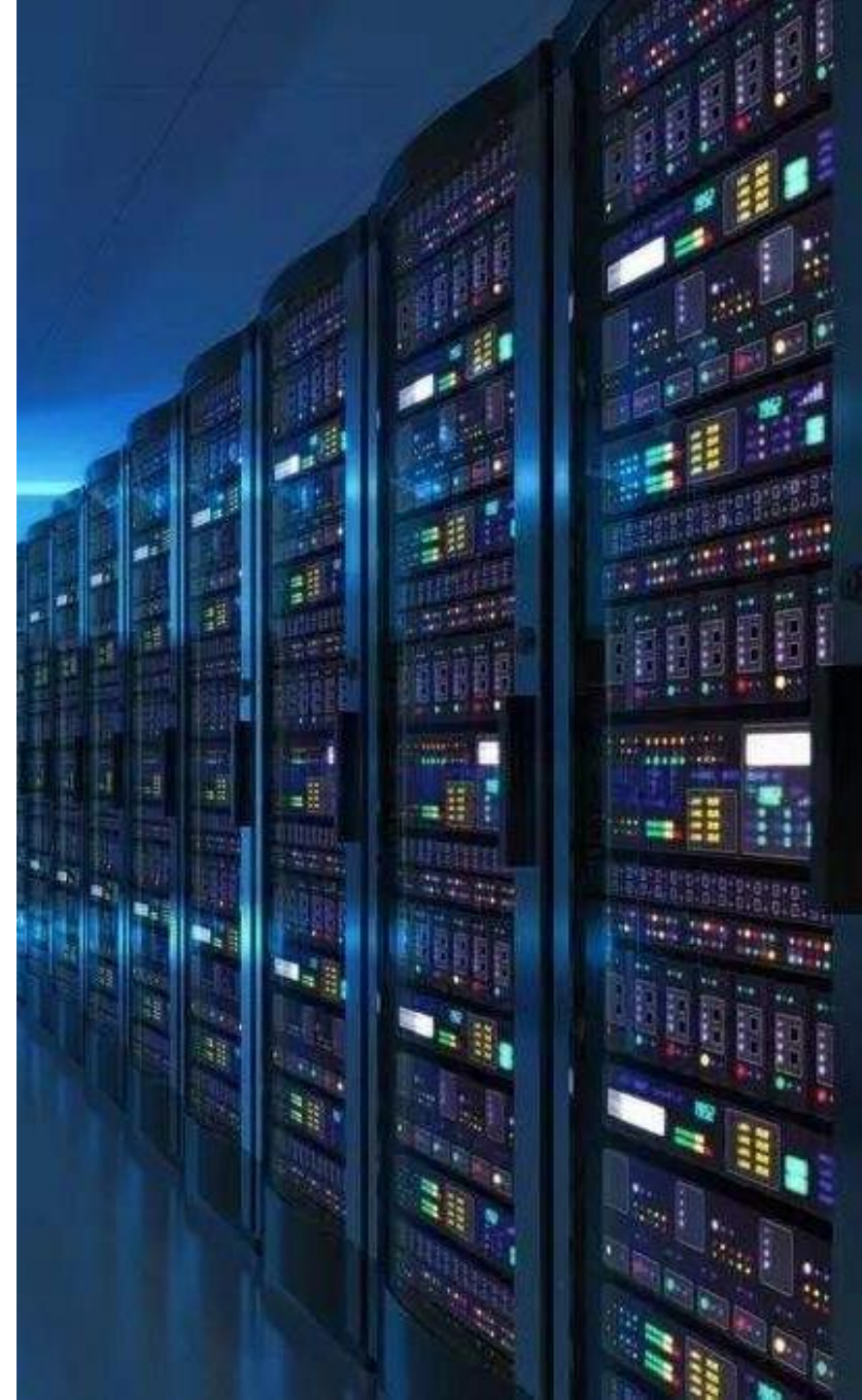
"Collection of independent computers"

- Multiple machines (Computers)
- Each can work alone
- Connected by a network

"Appears as a single system"

Users don't see the complexity

Like using Google - you don't know which server answers



WHY DISTRIBUTED?



The Problem Scenario:

"Let's imagine WhatsApp with 2 billion users on ONE computer:

Storage: Each user = 100MB of data

- $2 \text{ billion} \times 100\text{MB} = 200 \text{ petabytes}$
- No single computer has this much storage!

Processing: 100 billion messages per day

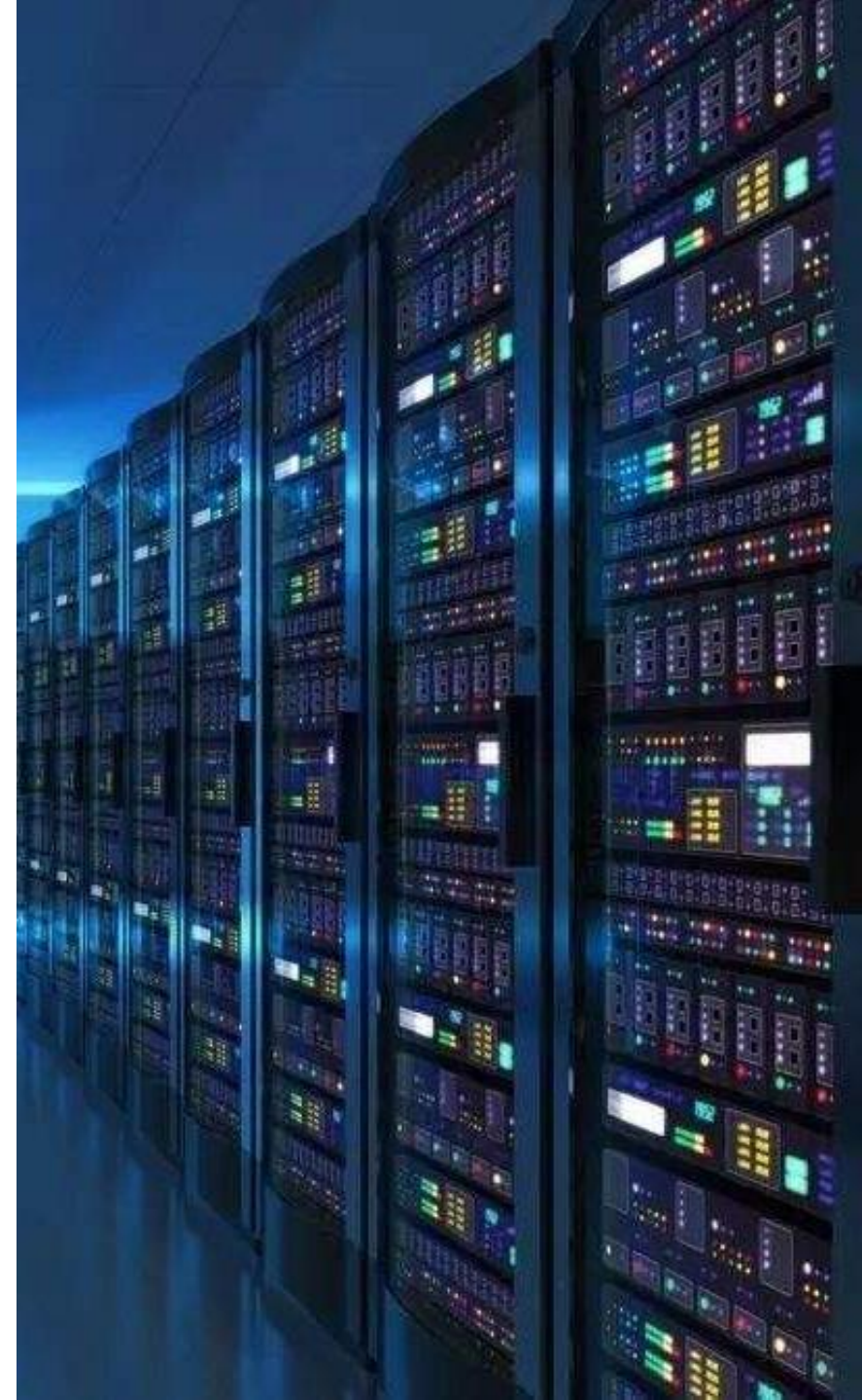
- That's 1.2 million messages per second
- One computer would crash instantly!

Availability: What if that one computer fails?

- 2 billion angry users!
- Business loses millions per minute"

WHY? KEY CONCEPTS

- **Scalability:** capacity of a system to add more resources without affecting the architecture.
- **Reliability:** Fault Tolerance
- **Latency**
- **Performance**
- **Cluster:**
 - **Node: MASTER and SLAVE**
 - **Load Balancing**
- **Algorithms: MapReduce**
- **Messaging:**
 - **Topic**
 - **Subscription**



WHY? EXAMPLES

Instagram:

- 2012: 30 million users
- 2023: 2 billion users

How? They added more servers, not bigger servers

Netflix:

Has copies of 'Stranger Things' on hundreds of servers

If 10 servers fail, you still watch your show

This is called 'fault tolerance'

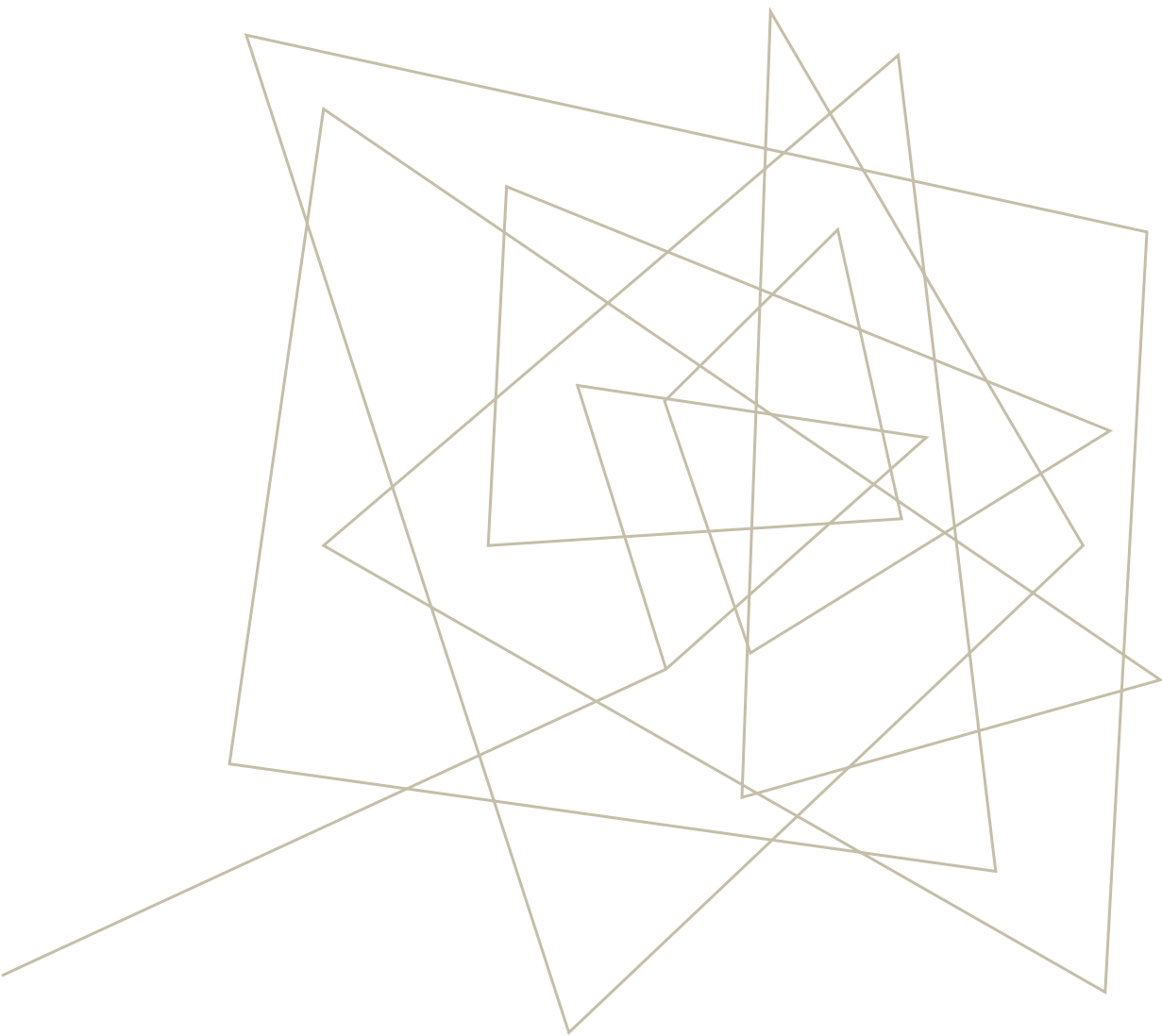
Google Search:

Your query goes to the nearest data center

Thousands of computers search simultaneously

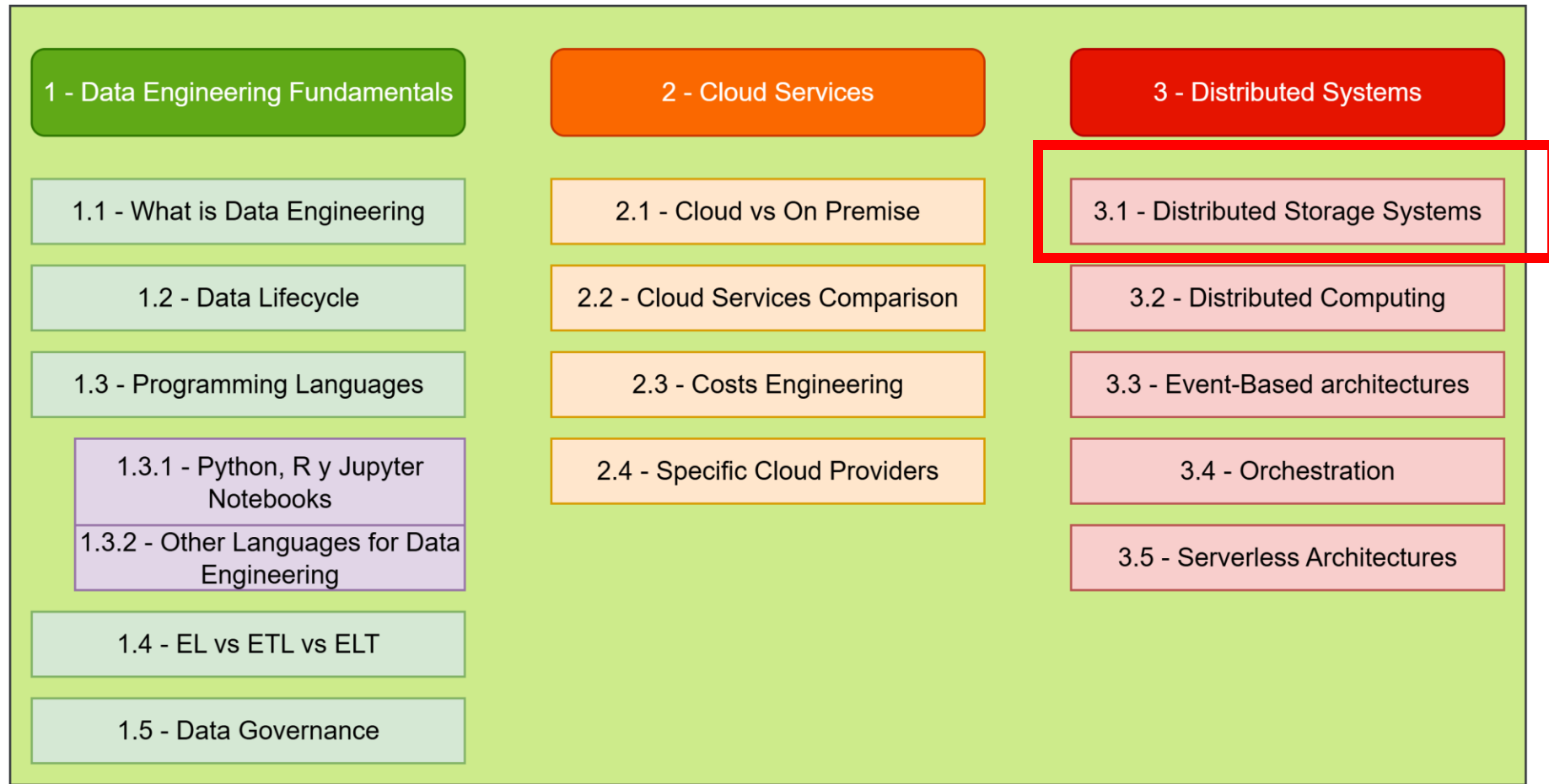
Results in 0.2 seconds

One computer would take hours!"



DISTRIBUTED STORAGE

SUBJECT CURRICULUM



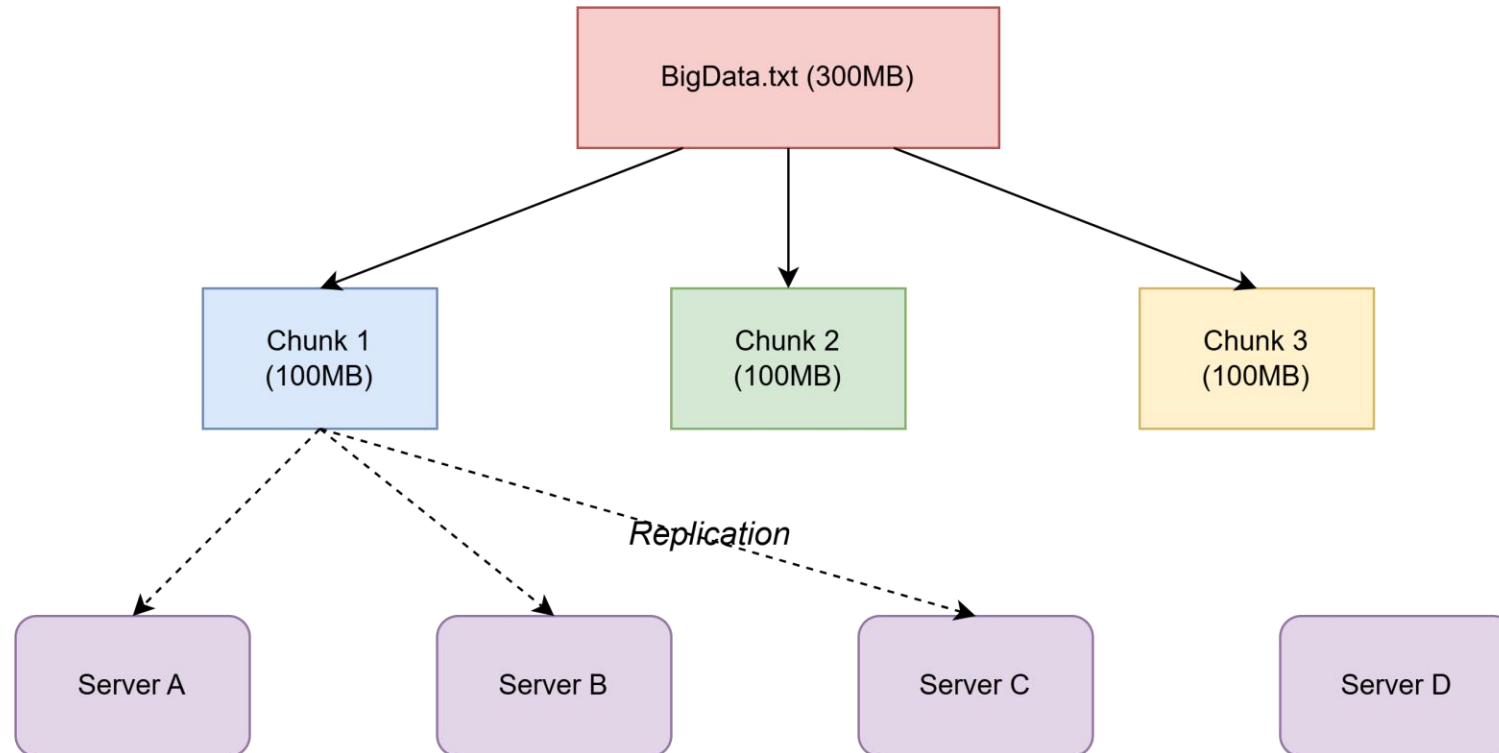
DISTRIBUTED STORAGE SYSTEMS

What is Distributed Storage?

Distributed storage is a method of storing data across multiple physical servers or locations, rather than keeping all data on a single machine. The data appears as one unified system to users, even though it's spread across many devices.

Simple Analogy: Think of it like storing your belongings in multiple lockers instead of one giant closet. If one locker breaks, you still have access to items in the others.

WHY DISTRIBUTED?



Instagram Photo Upload Journey

2

Data Center Europe (Primary)

Replicate

Data Center Americas

Edge Server Mexico City

3

View photo

Friend (Mexico)

Mexico City

4

View photo

Friend (Japan)

Tokyo, Japan

- 🌟 All happens in < 500ms!
- 📷 Same photo, different locations
- 🌐 Closest server responds
- 💾 3 copies for reliability

1

You (Madrid)

Madrid, Spain



Upload photo

Load Balancer

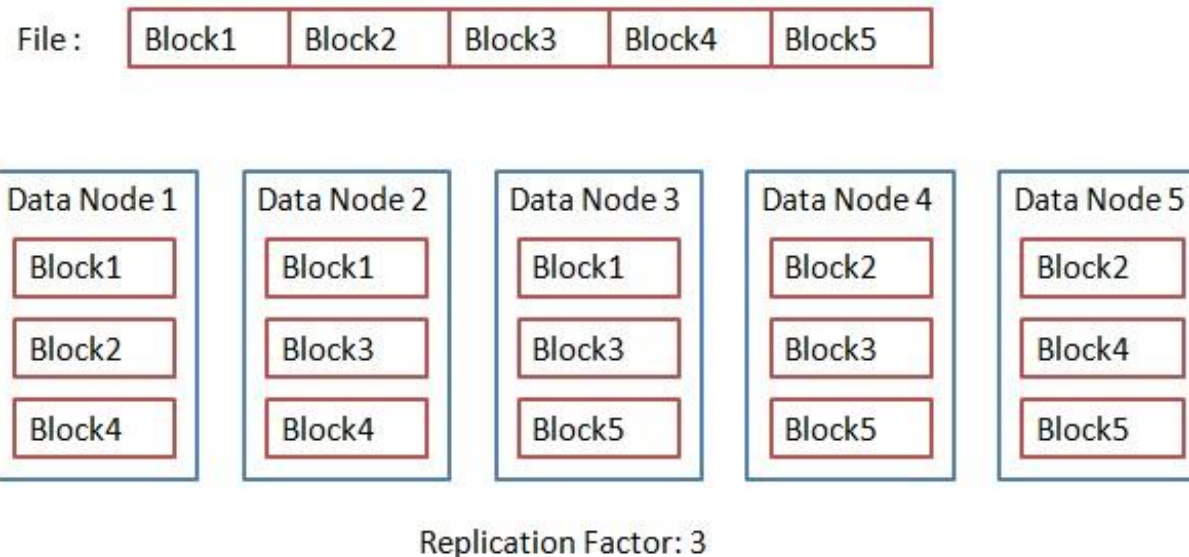
Data Center Asia

Edge Server Tokyo

Timeline: 0ms ————— 200ms ————— 500ms

REPLICATION FACTOR

The replication factor is the number of copies (replicas) of data that a distributed system maintains across different nodes or locations. It's a fundamental concept in distributed systems that directly impacts reliability, availability, and performance.

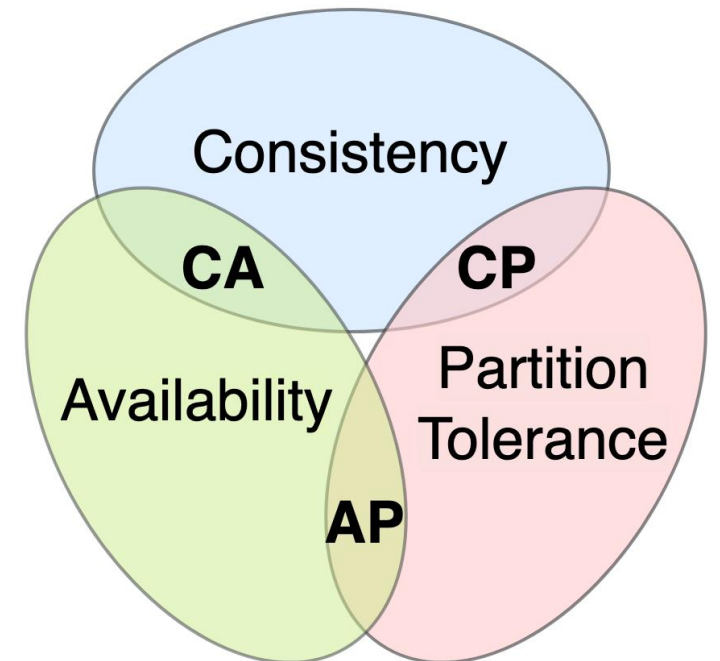
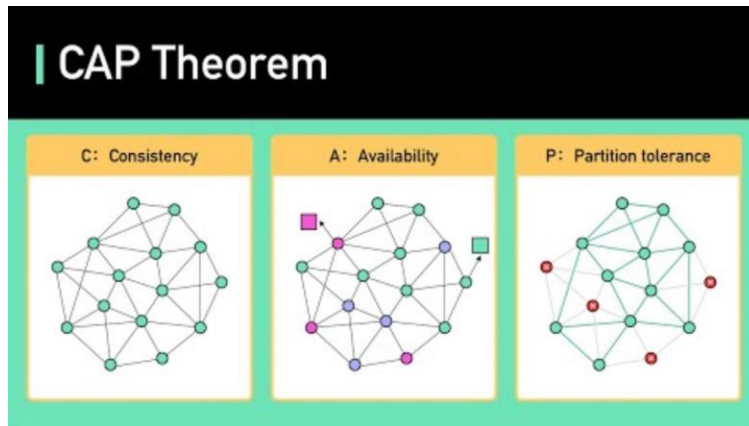


CAP THEOREM

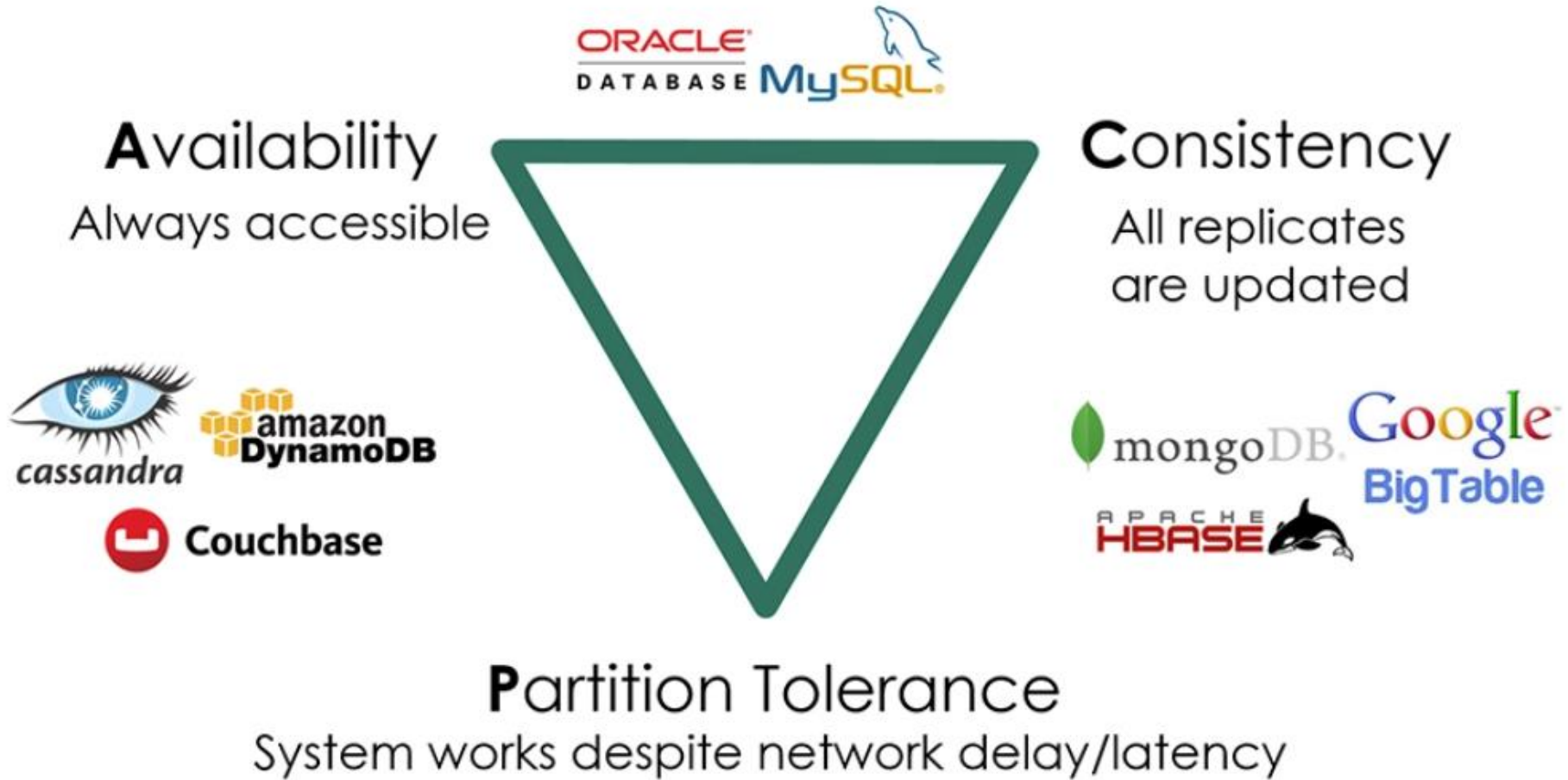
The CAP Theorem, also known as Brewer's Theorem (proposed by Eric Brewer in 2000), states that in a distributed data system, it is impossible to simultaneously guarantee all three of the following properties:

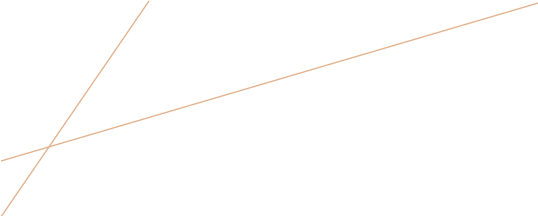
- Consistency: All nodes see the same data at the same time.
- Availability: The system remains operational 100% of the time.
- Partition Tolerance : The system continues to operate despite network failures.

You can only guarantee 2 out of 3 at any given time.



CAP THEOREM





CAP THEOREM

What does each letter in CAP stand for?

Explain what "Consistency" means in distributed systems

Give an example of "Availability" from daily life

What is a "network partition"?

CAP THEOREM: TRUE OR FALSE



A distributed system can have all three CAP properties



Partition tolerance is optional in distributed systems



CA systems are common in modern cloud applications



Banks typically choose AP systems



HDFS is a distributed file system designed to store very large files across multiple machines, while appearing as a single file system to users.

NameNode: The "Brain" of HDFS

- Manages file system namespace
- Stores metadata (not actual data)
- Keeps track of file locations
- Single point of failure (addressed by HA)

Like: A library catalog

- Knows every book's location
- Doesn't store the books

DataNode: The "Storage Workers"

- Store actual data blocks
- Report to NameNode regularly
- Execute read/write requests
- Handle block replication

Like: Library shelves

- Actually hold the books
- Tell catalog their contents

SecondaryNameNode: The "Assistant"

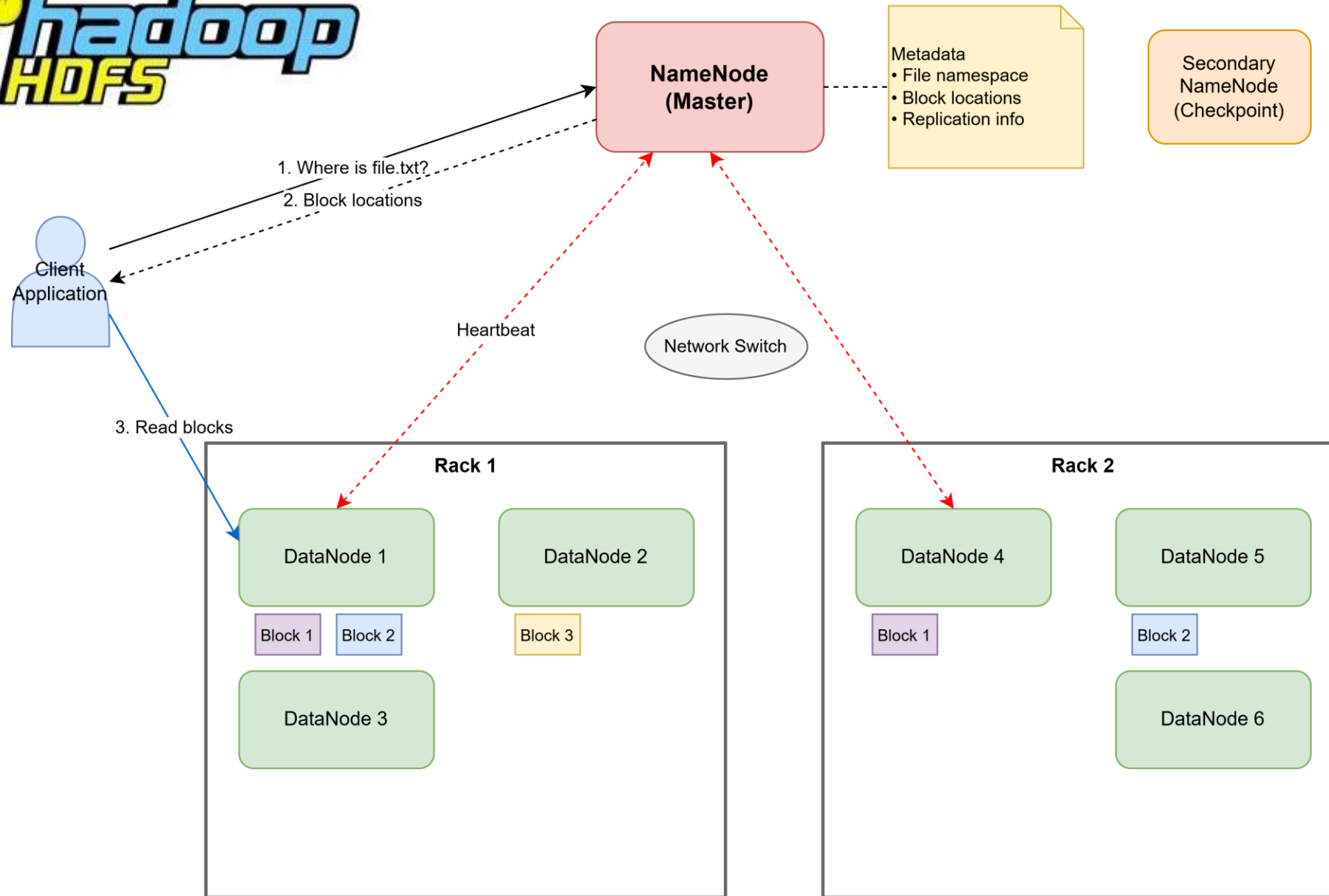
- NOT a backup NameNode!
- Helps with checkpointing
- Merges edit logs
- Reduces NameNode restart time

Like: An assistant librarian

- Helps organize the catalog
- Makes backup copies



HDFS (Hadoop Distributed File System) Architecture



HDFS Write Process:

- 1 Client requests write
- 2 NameNode allocates blocks
- 3 Client writes to pipeline
- 4 DataNodes replicate
- 5 Acknowledge to client

Key HDFS Features:

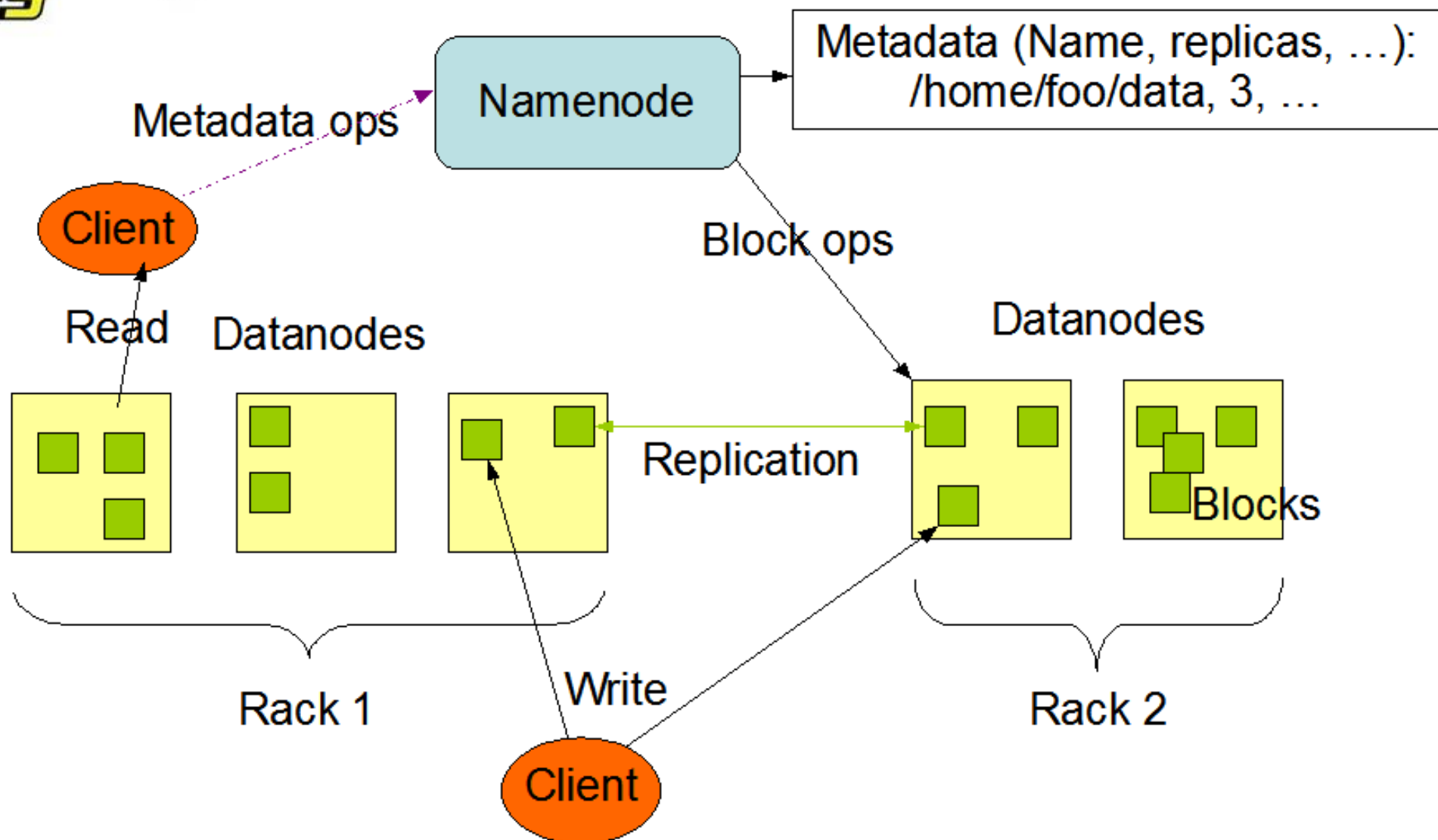
- Default block size: 128MB
- Replication factor: 3
- Write once, read many
- Rack awareness
- Automatic failover

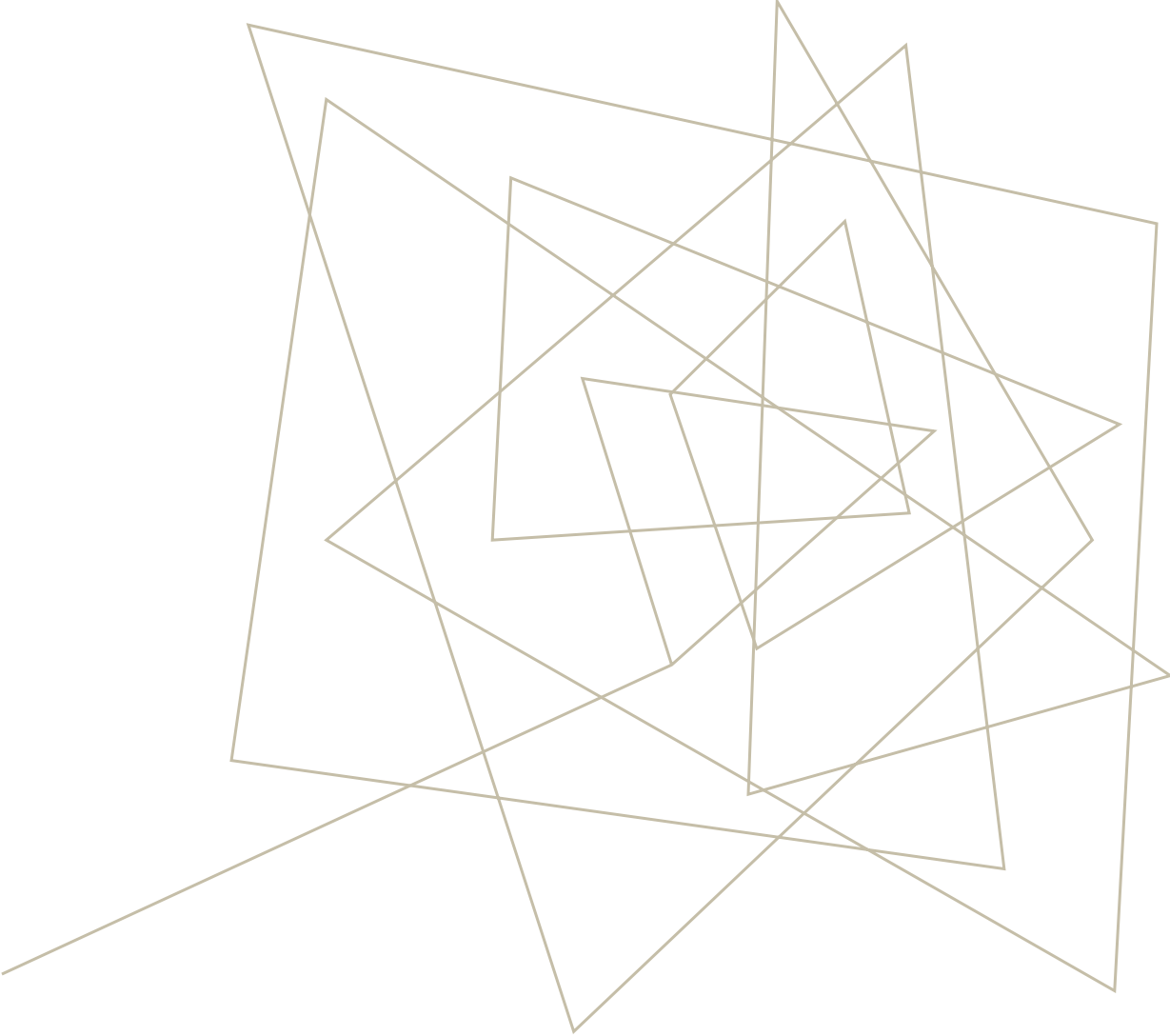
Legend:

- Data flow
- Metadata/Control



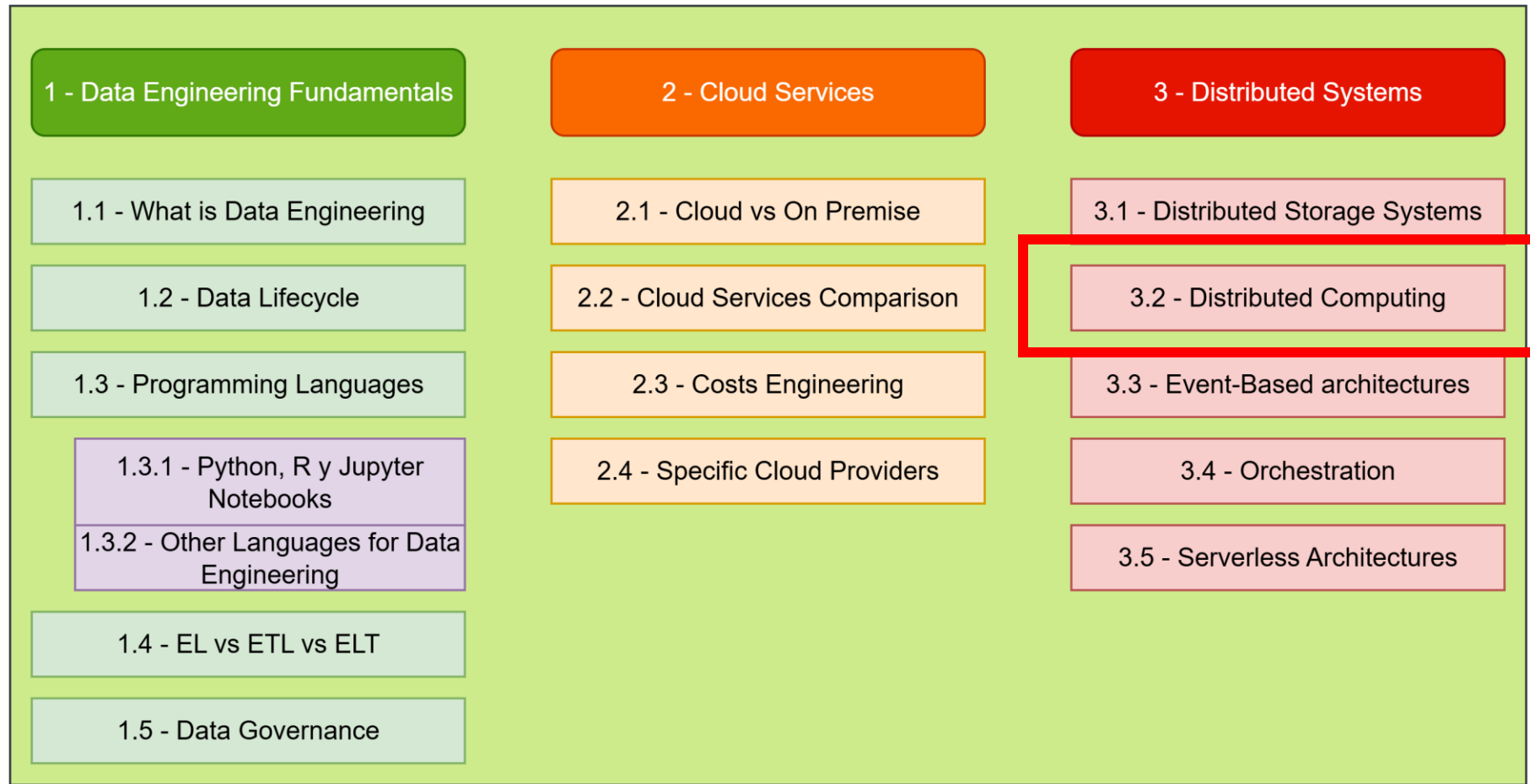
HDFS Architecture





DISTRIBUTED COMPUTING

SUBJECT CURRICULUM



DISTRIBUTED COMPUTING

How do we **process** distributed data efficiently?

Remember: distributed storage

- Data partitioning across nodes
- Replication for fault tolerance
- HDFS, distributed databases
- CAP theorem considerations



KEY CONCEPTS

1. Data Locality

Moving computation to where data resides rather than transferring data across the network. Principle: "Moving computation is cheaper than moving data."

2. Parallelization

Breaking tasks into independent subtasks that execute simultaneously across multiple nodes to reduce total execution time.

3. Fault Tolerance

System's ability to continue operating despite node or task failures through mechanisms like task re-execution, data replication, and lineage tracking.

4. MapReduce Paradigm

Programming model with two phases: Map (transform data independently in parallel) and Reduce (aggregate results by key), abstracting distribution complexity from programmers.

5. Partitioning

Dividing data into chunks distributed across nodes using strategies like hash partitioning, range partitioning, or custom partitioning to optimize data locality and performance.

KEY CONCEPTS

6. Lazy Evaluation

Building an execution plan without immediate execution; operations are only triggered by actions, enabling pipeline optimization and minimizing unnecessary data movement.

7. Shuffle

Redistributing data across nodes (typically by key) between processing stages. Most expensive operation due to network and disk I/O costs.

8. In-Memory Computing

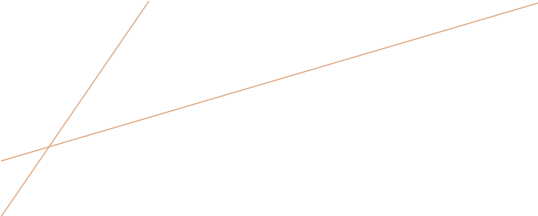
Keeping data in RAM between operations instead of writing to disk, achieving ~100x performance improvement, critical for iterative algorithms.

9. Lineage

Tracking the sequence of transformations applied to create a dataset, enabling efficient fault tolerance by recomputing lost data instead of replicating it.

10. Master-Worker Pattern

Architectural pattern where a master/coordinator node distributes work and monitors progress while worker/executor nodes perform assigned tasks and report status.



Distributed Storage	Distributed Computing
WHERE to store data	HOW to process data
Data placement	Task placement
Replication	Parallelization
Focus: Availability	Focus: Performance

A task can be seen as a code file or algorithm used to modify the data.



PRINCIPLES

1. Data Locality

- Process data where it's stored
- Minimize network transfer
- "Moving computation is cheaper than moving data"

2. Parallelization

- Divide task into subtasks
- Execute simultaneously
- Aggregate results

3. Fault Tolerance

- Nodes will fail
- System must continue
- Re-execute failed tasks

MAP REDUCE PARADIGM

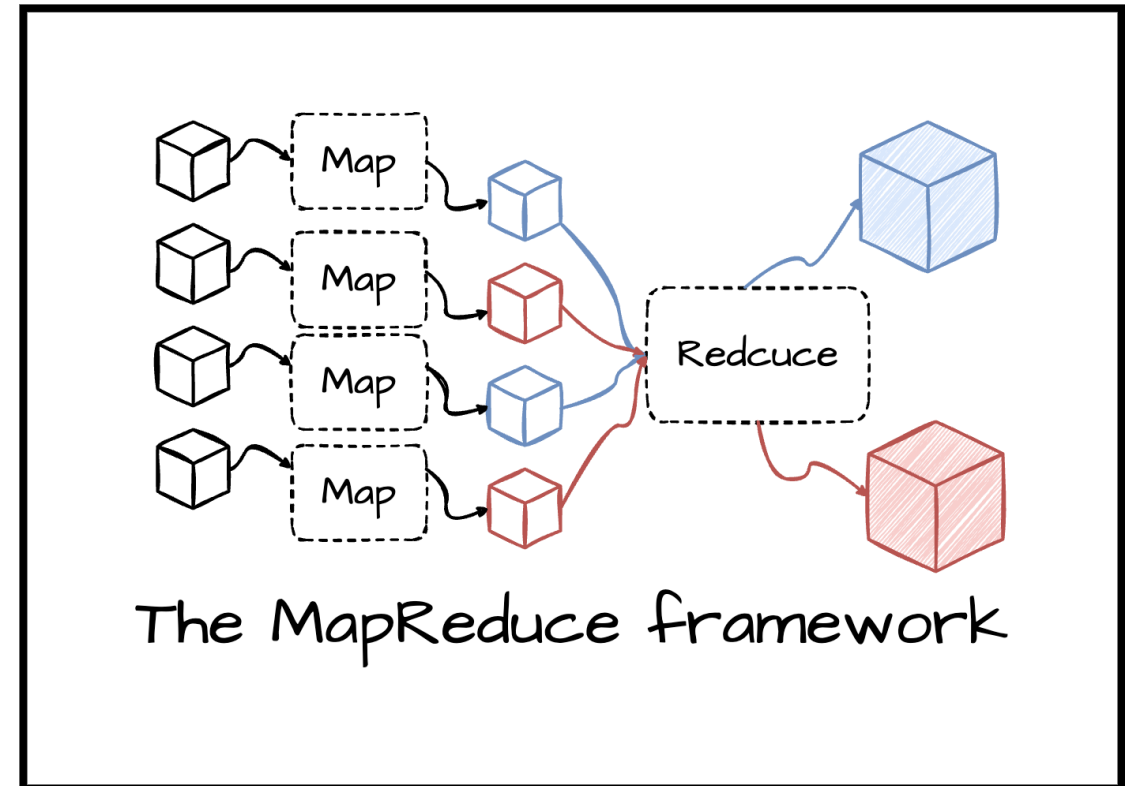
Origin: Google's solution for distributed computing (2004): <https://research.google/pubs/mapreduce-simplified-data-processing-on-large-clusters/>

Two main phases:

- 1.Map:** Apply function to data chunks independently
- 2.Reduce:** Aggregate results from map phase

Keys

- Abstraction hides complexity
- Automatic parallelization
- Built-in fault tolerance



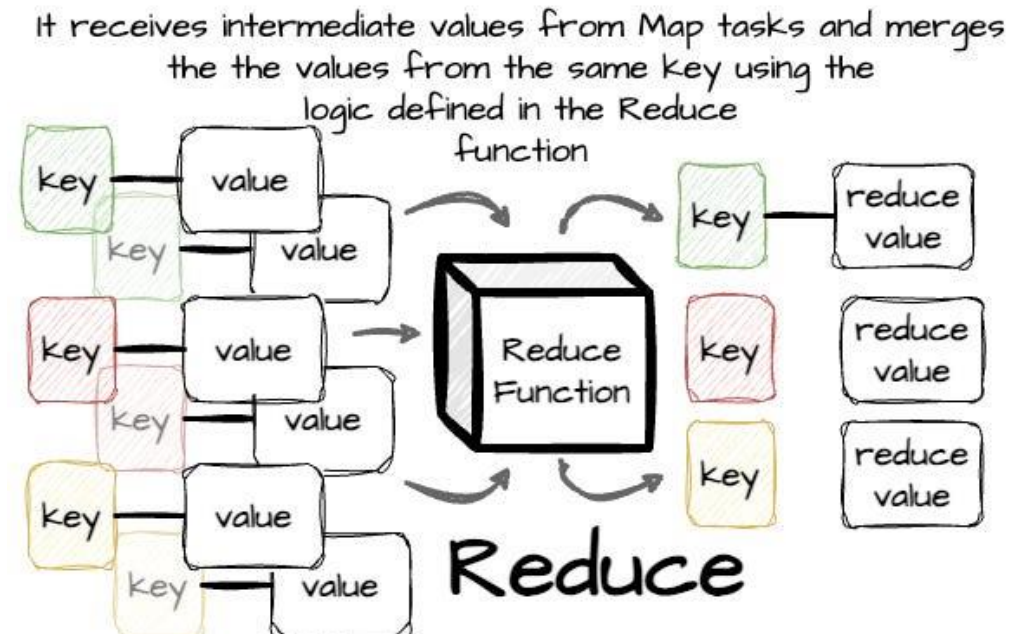
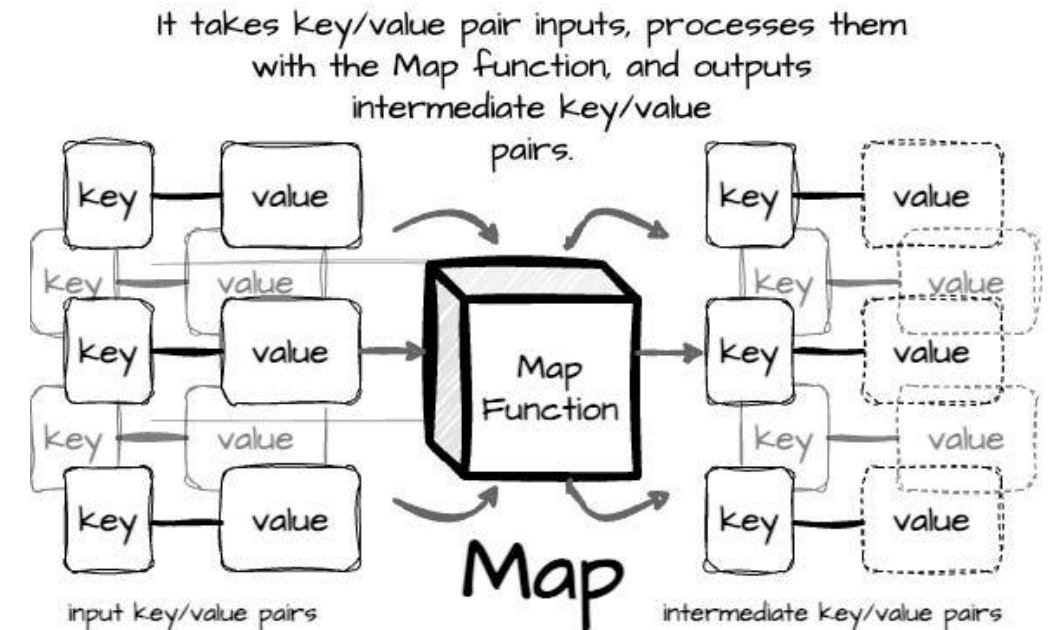
MAP REDUCE PARADIGM

- Map:** It takes key/value pair inputs, processes them, and outputs intermediate key/value pairs. The library will group all values of the same key and pass them to the Reduce tasks.

- Reduce:** It receives intermediate values from Map tasks. The intermediate values are supplied to the Reduce via an iterator. It then merges the intermediate values from the same key using the logic defined in the Reduce function (e.g., Count, Sum, ...). The Reduce typically produces at most one output value.

Master Node: Coordinates task assignment

Worker Nodes: Execute map/reduce tasks



MAP REDUCE PARADIGM

Challenges:

- Which worker processes which data partition?
- What if a worker fails mid-task?
- How to handle stragglers (slow workers)?

Solutions:

- **Data locality:** Assign tasks to nodes with data
- **Speculative execution:** Re-run slow tasks elsewhere
- **Task redundancy:** Backup tasks for critical operations

Limitations:

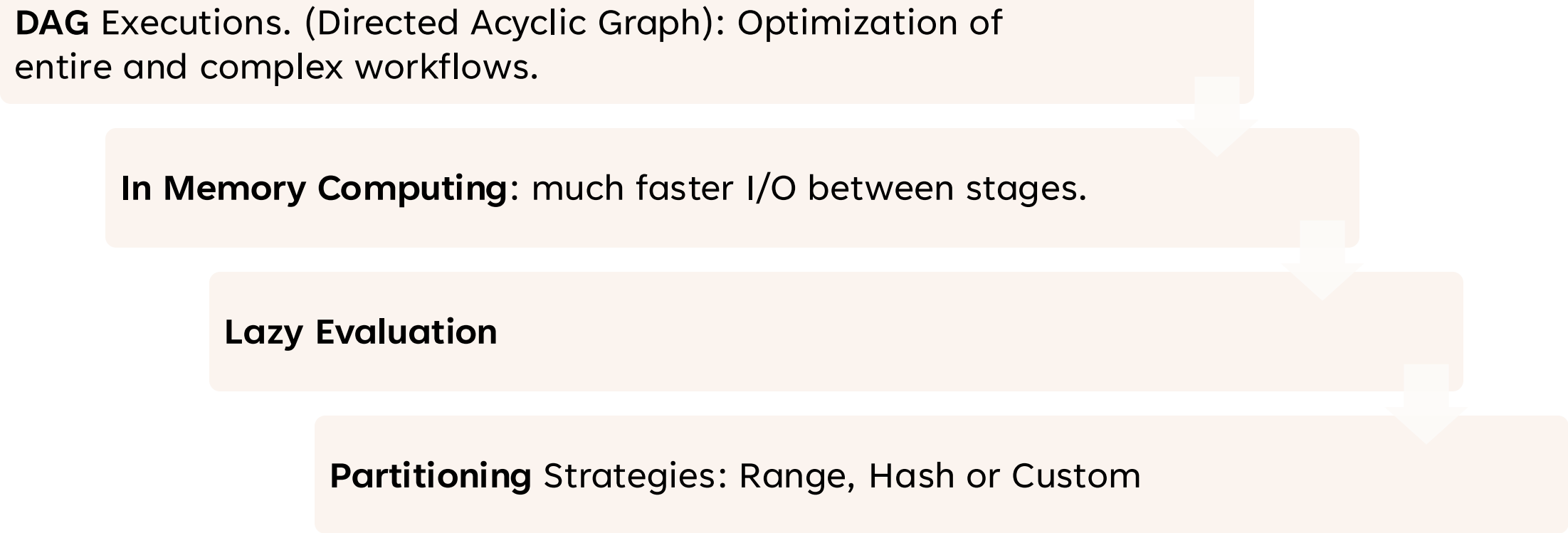
- ❌ Disk I/O between stages
- ❌ Not suitable for iterative algorithms
- ❌ High latency (batch processing)
- ❌ Complex multi-stage jobs are cumbersome
- ❌ No interactive queries

Timeline:

- 2004: MapReduce paper (Google)
- 2006: Hadoop MapReduce (Yahoo)
- 2010: Limitations become clear
- 2014: Apache Spark gains adoption
- 2020+: Specialized frameworks

BEYOND MR

DAG Executions. (Directed Acyclic Graph): Optimization of entire and complex workflows.



In Memory Computing: much faster I/O between stages.

Lazy Evaluation

Partitioning Strategies: Range, Hash or Custom

APACHE HADOOP OVERVIEW

Core components:

1.HDFS (Storage)

1. You already know this
2. Foundation for everything else

2.YARN (Resource Management)

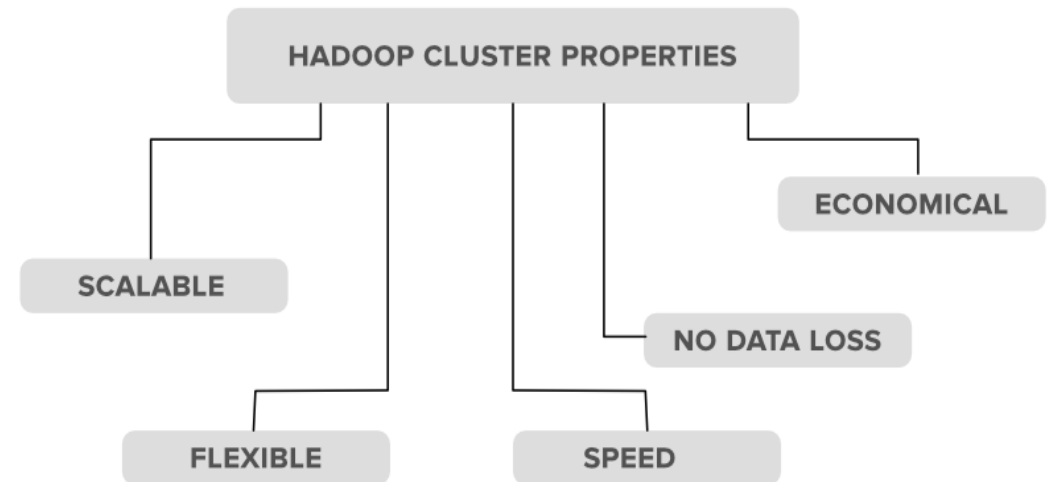
1. Cluster resource manager
2. Schedules applications

3.MapReduce (Computing)

1. Original processing framework
2. Batch-oriented

4.Ecosystem Tools

1. Various specialized tools



CLOUDEXERA

YARN

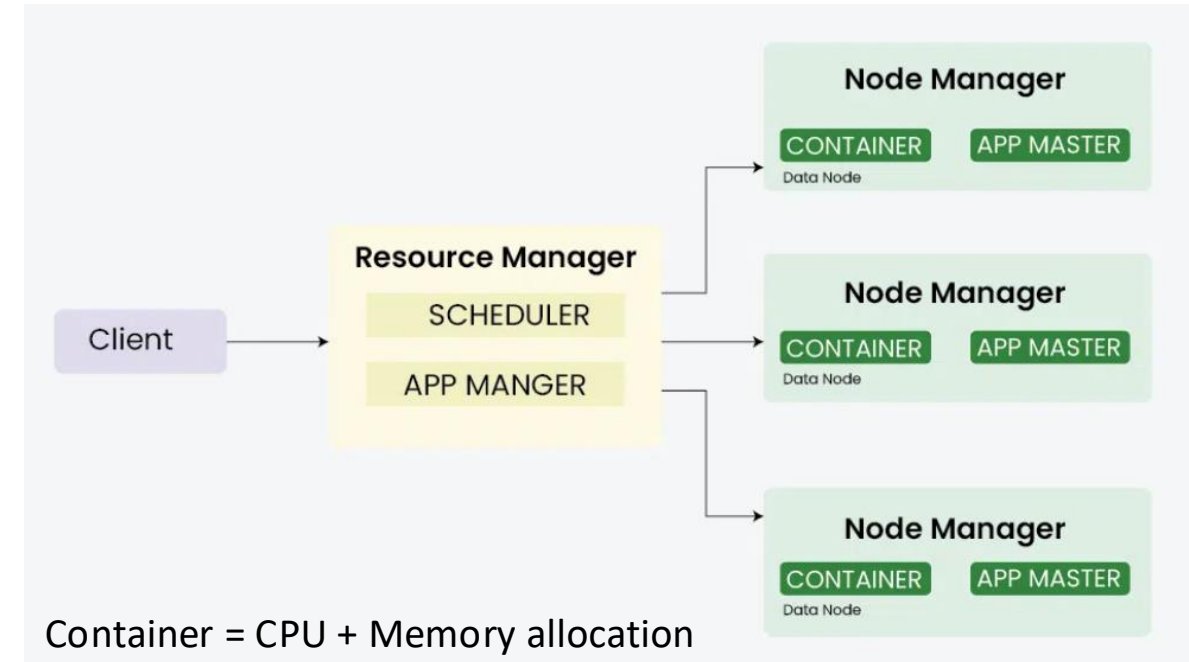
YET ANOTHER RESOURCE NEGOTIATOR

Architecture:

- **ResourceManager:** Cluster-wide resource allocation
- **NodeManager:** Per-node resource management
- **ApplicationMaster:** Per-application coordination

YARN Workflow:

1. Client submits application to ResourceManager
2. RM allocates container for ApplicationMaster
3. AM requests containers for tasks
4. RM allocates containers on NodeManagers
5. AM launches tasks in containers
6. Tasks execute and report progress
7. AM reports to RM, client monitors



Separated from processing: not locked to MapReduce





CREATE A AND MONITOR A SINGLE NODE HADOOP CLUSTER

<https://rancavil.medium.com/mapreduce-example-with-python-b435a9858718>

APACHE SPARK

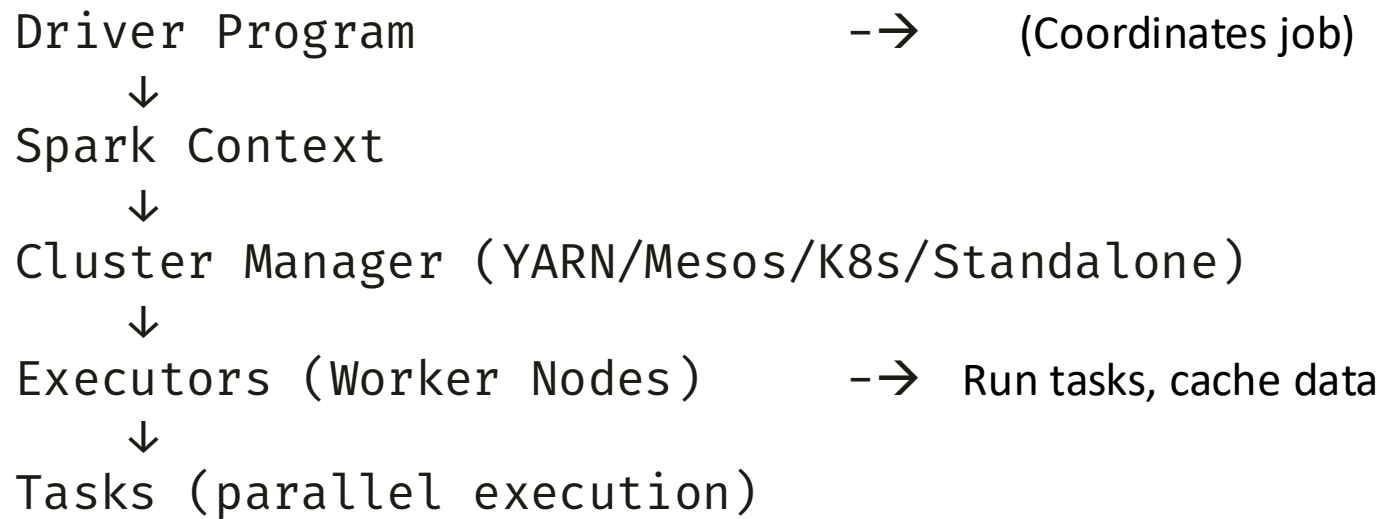


- **Created:** 2009 at UC Berkeley
- **Goal:** Address MapReduce limitations
- **Key innovation:** In-memory computing
- **Performance:**
 - Up to 100x faster than Hadoop MapReduce (in-memory)
 - Up to 10x faster on disk

Feature	Hadoop MapReduce	Apache Spark
Speed	Slower (disk-based)	Faster (memory)
Ease of use	Verbose Java	Concise Python/Scala
Processing	Batch only	Batch, streaming, ML
Latency	Minutes-Hours	Seconds-Minutes
Fault tolerance	Task re-execution	RDD lineage
Interactivity	No	Yes (shell)

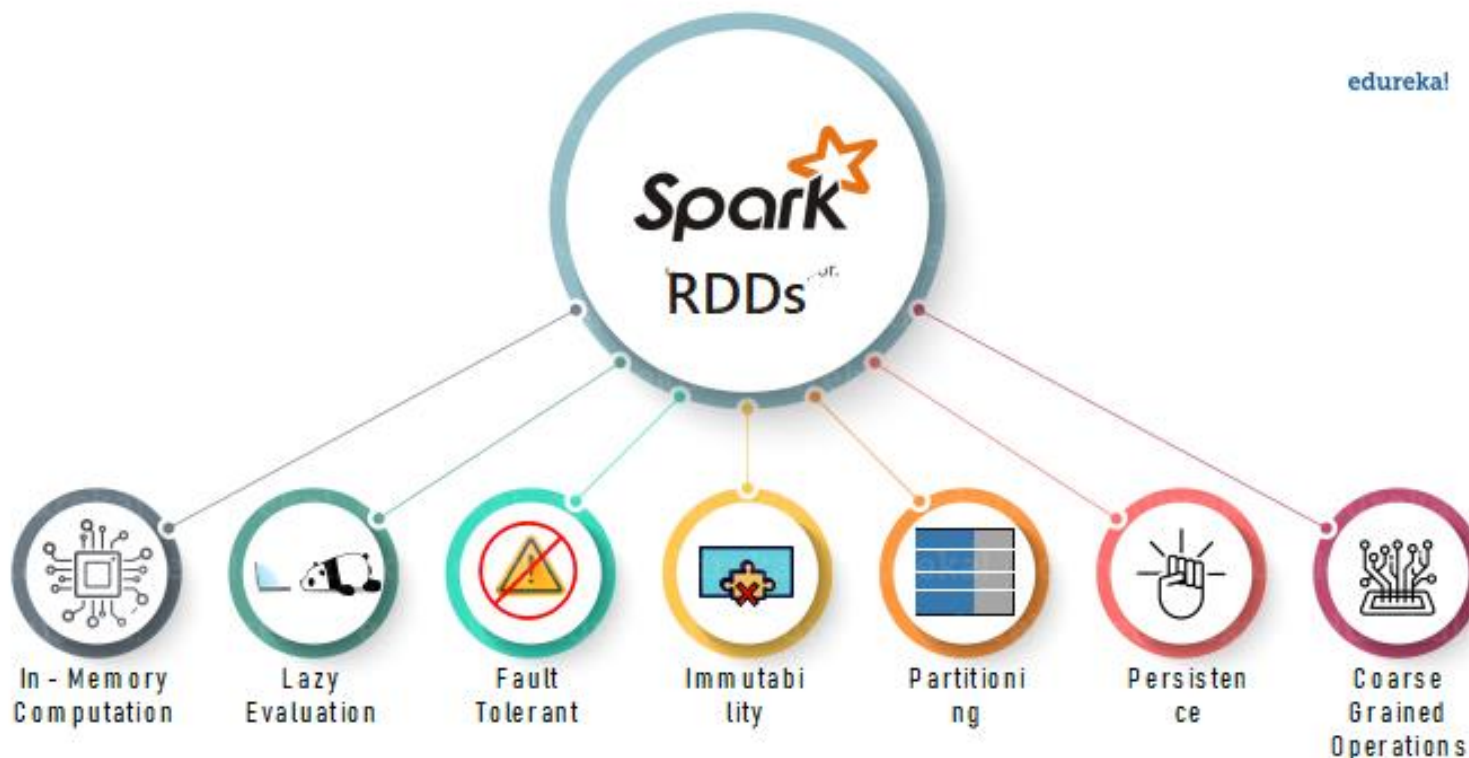
APACHE SPARK

Workflow



APACHE SPARK: RDD

RDD or (**Resilient Distributed Data set**) is a fundamental **data structure** in Spark. The term **Resilient** defines the ability that generates the data automatically or data **rolling back** to the **original state** when an unexpected calamity occurs with a probability of data loss.

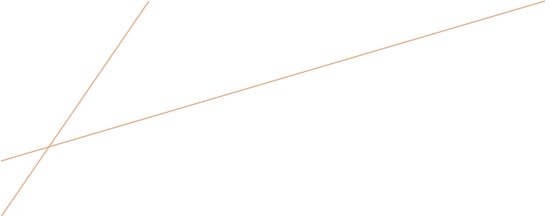


Transformations (lazy):

- `map()`: Apply function to each element
- `filter()`: Select elements
- `flatMap()`: Map then flatten
- `reduceByKey()`: Aggregate by key
- `join()`: Combine datasets

Actions (execute):

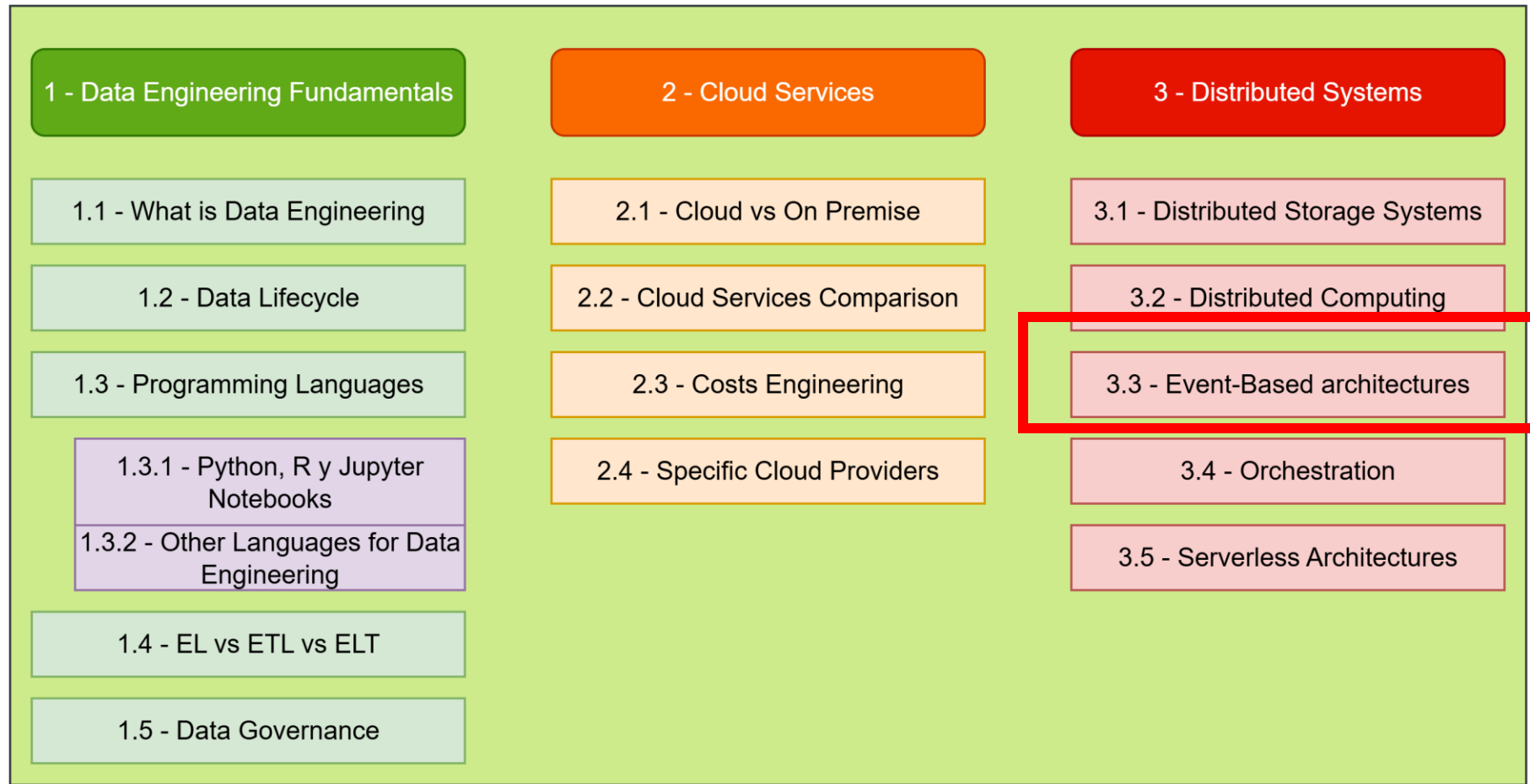
- `count()`: Count elements
- `collect()`: Return all data
- `reduce()`: Aggregate all elements
- `save()`: Write to storage
- `take()`: Return first n elements



AZURE DATABRICKS LAB

<https://microsoftlearning.github.io/mslearn-databricks/Instructions/Exercises/LA-01-Explore-Azure-Databricks.html>

SUBJECT CURRICULUM



EDA: EVENT DRIVEN ARCHITECTURES

REAL-TIME DATA SYSTEMS



EDA: EVENT DRIVEN ARCHITECTURES

REAL-TIME DATA SYSTEMS

- Understand event-driven architecture
- Compare Kafka, Pub/Sub, and message queues
- Design your own event-based system
- Apply concepts to data engineering scenarios



EDA: EVENT DRIVEN ARCHITECTURES

REAL-TIME DATA SYSTEMS

Quick Check

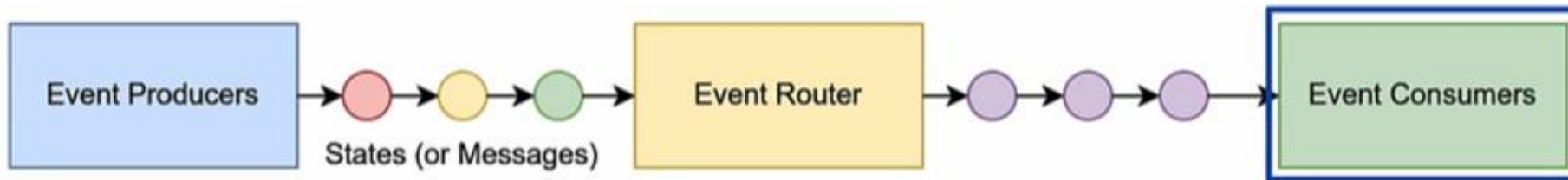
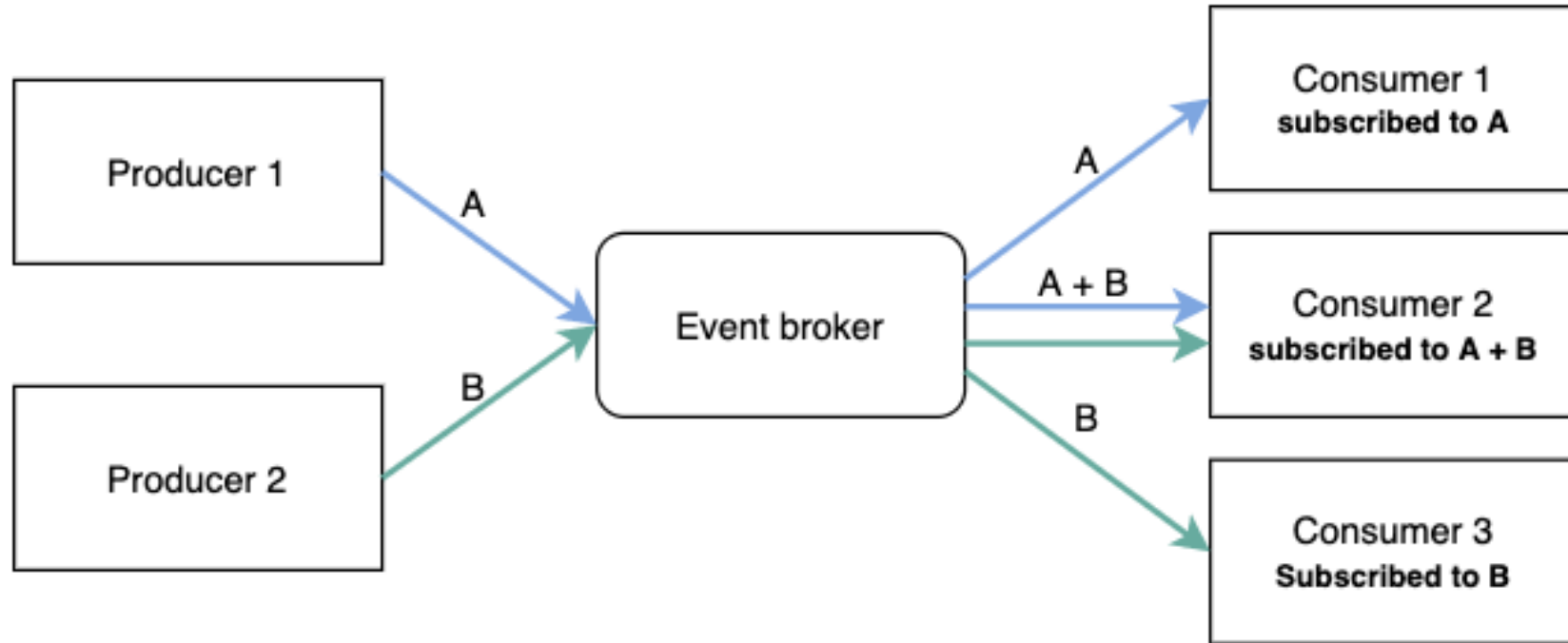
- Have you used any messaging app today?
- What you know about micro services?
- Heard of Apache Kafka?
- Know what "real-time" means in tech?



EDA: KEY CONCEPTS

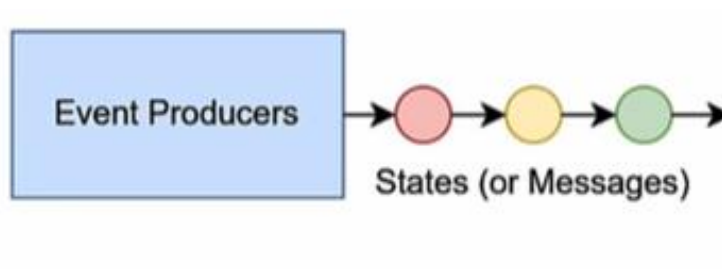
- **1. Event:** A significant change in state or an occurrence that happens at a specific point in time within a system.
- 2. Message:** A discrete unit of data that is transmitted from one component to another, containing information about an event or a command to be executed.
- **3. Producer (Publisher) :** A component or application that creates and sends messages/events to a messaging system.
- **4. Consumer (Subscriber) :** A component or application that receives and processes messages/events from a messaging system.
- **5. Broker (Message Broker or Router) :** An intermediary program/server that translates, routes, and delivers messages between producers and consumers.
- 6. Topic:** A named logical channel or category to which messages are published and from which consumers can subscribe to receive messages.
- 7. Queue:** A data structure that stores messages in a first-in-first-out (FIFO) order, where each message is typically processed by only one consumer.

EDA: CORE COMPONENTS



EDA: PRODUCERS

CREATES and SENDS the events.



- IoT sensors 📡
- User clicks 🖱️
- Database changes 💾
- API calls 🖨️

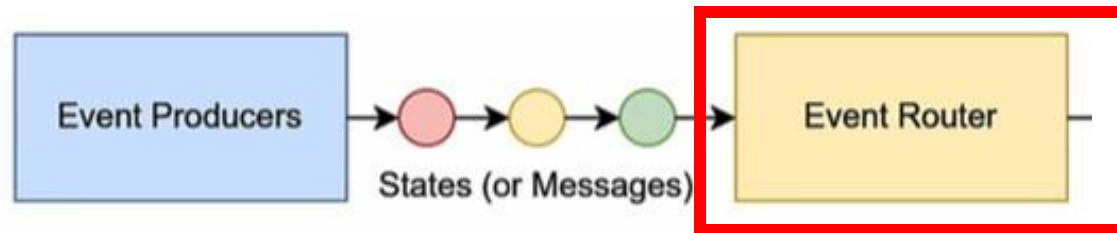
python

```
producer.send('user-events',  
              {'user_id': 123,  
               'action': 'click'})
```

EDA: BROKERS

THE TRAFFIC CONTROLLERS

The main part of the architecture. Receives the messages and handles delivery.

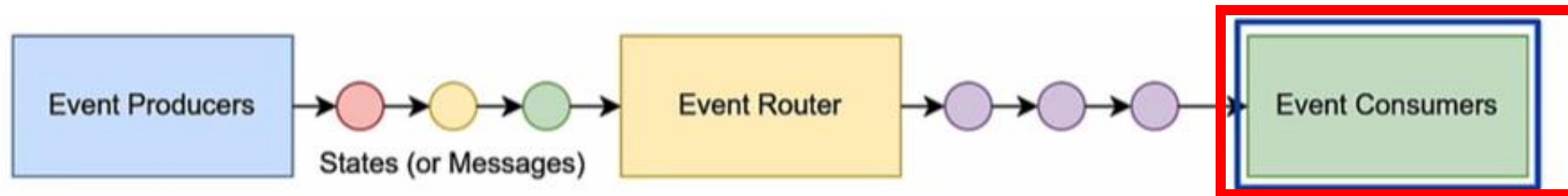


- IoT sensors 📡
- User clicks 🖱️
- Database changes 💾
- API calls 🔌

- Store messages
- Route to consumers
- Ensure delivery
- Handle failures

EDA: CONSUMERS

The main part of the architecture. Receives the messages and handles delivery.



- IoT sensors 🗝️
- User clicks 🖱️
- Database changes 💾
- API calls 🔌

- Analytics service 📊
- Database writer 💾
- Notification sender 📧
- ML pipeline 🤖

CLASSIC VS EVENT DRIVEN

CLASSIC: Request - Response

Whatch this video

<https://www.youtube.com/watch?v=gNJb6WSyzP4>

What are the PROS and CONS of each type of data exchange? Two Groups. Debate, search information, select a 'leader' that will explain the conclusion. 5 minutes!

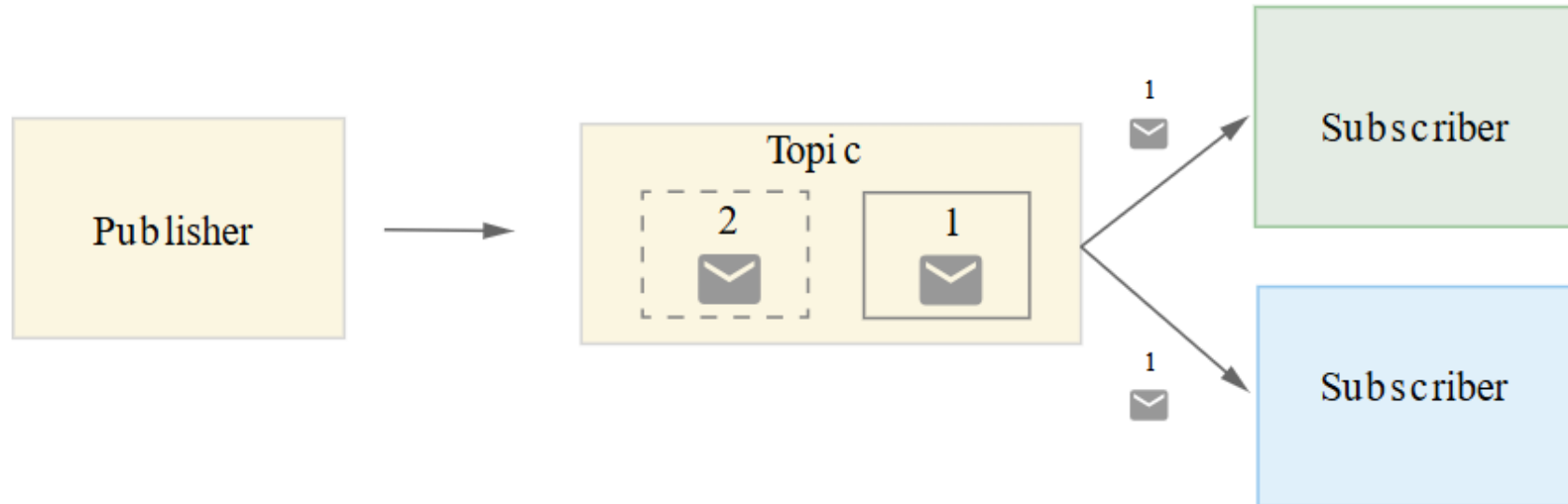
EDA: PUB/SUB PATTERN

Same message replicates to multiple consumers.

Subscription pattern.

Topic: abstraction or classification of the messages.

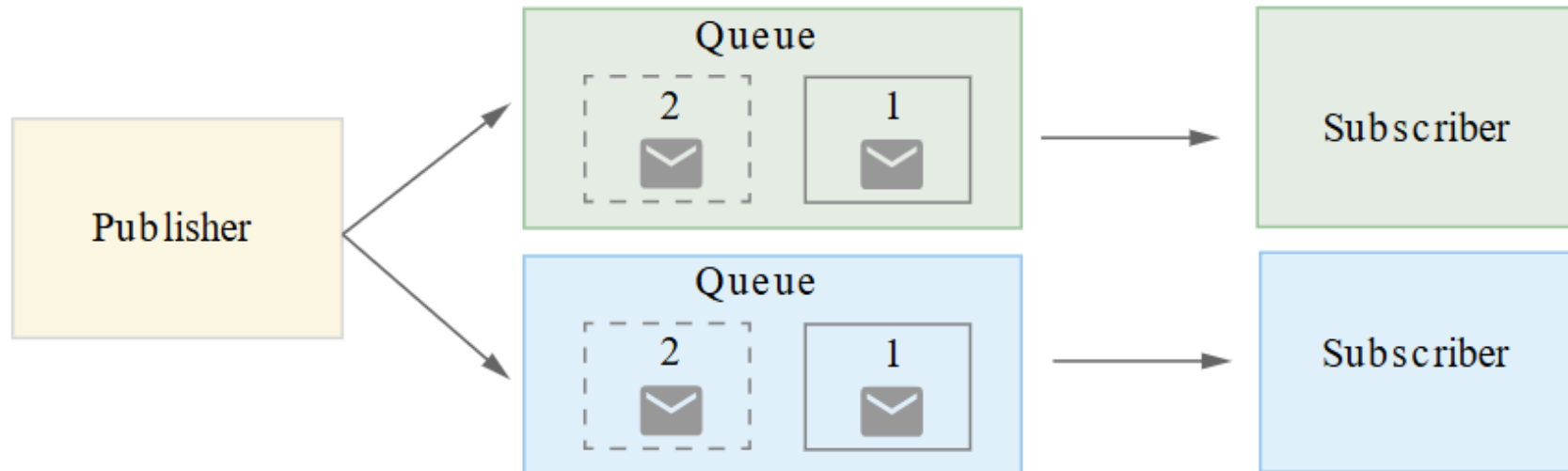
Example: youtube channel. Creator publishes a video -> many subscribers are notified.



MESSAGE QUEUE PATTERN

Task Distribution:

- Publisher sends to different classified queues.
- Queues send one by one to all subscriber.
- Example: mailing lists.

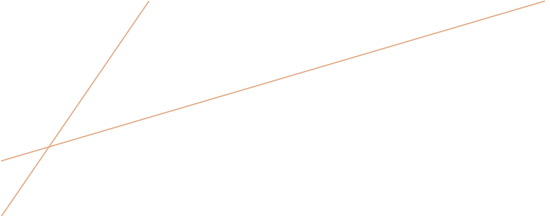


HANDS ON: DESIGN EVENT ARCHITECTURE (PUB/SUB)

- Choose an app or service from these at the right
- Find a minimum of six events you want to track
- Design an event driven infrastructure that takes in count the following:
 - Identify **Consumers** and **Publishers**.
 - Define at least a **TOPIC**
 - The events should be consumed by other users, a central app, or both.
 - You must consider both Analytical purposes (a database) and application purposes (other users).

EXAMPLE: 'Add to Cart' event, 'Play' event, 'Subscribe channel' event.





AZURE DATABRICKS LAB

<https://microsoftlearning.github.io/mslearn-databricks/Instructions/Exercises/LA-01-Explore-Azure-Databricks.html>